

树与二叉树:

- 是一对多的一种数据结构
- (介于线性表(一对一)和图(多对多)之间)

eg: 

●: 叶子
●: 分支结点
●: 根结点

- 性质: 除根结点外, 均有唯一的前驱; 除叶子外, 均有任意个(不为0)后继
- 有根树定义: n 个结点 ($n \geq 0$) 的有限集合, $n=0$ 时为空树, 记作 $T = \{ \emptyset, T_1, T_2, \dots, T_m \}$ ($n=0$)
T的根结点 根的子树
(每个子树的根结点的唯一前驱) (子树之间无相交)

- 概念: 子女、父、兄弟、度 (结点的子女个数)、树的度 (各个结点的度的最大值)

分支结点 (度不为0的非根结点)、叶结点 (度为0的结点)、祖先、子孙

结点的层次 (规定根结点为第1层, 其子女结点等于其层次+1, 依次进行下去)

深度 (结点的深度即为其层次, 树的深度即为离根最远的结点的深度)

高度 (规定叶结点的高度为1, 其双亲结点高度为其高度+1, 树的高度即为根结点的高度)

有序/无序树 (各个结点的顺序是否重要)、森林 (m 个树组成的集合)

- 二叉树: 各个结点最多有两个子女 (称为左子树、右子树)

分为5大类: 

- 概念: 满二叉树 (深度为 k 且共有 $2^k - 1$ 个结点的二叉树, 每个非叶结点的度均为2)

完全二叉树 (满二叉树的最底层, 可以从最右边的叶结点开始连续向左缺结点, 其他不能缺)

- * 性质1: 若二叉树结点的层次从1开始, 则其第 i 层最多有 2^{i-1} 个结点 ($i \geq 1$)

- * 性质2: 深度为 k 的二叉树最少有 k 个结点, 最多有 $2^k - 1$ 个结点 ($k \geq 1$)

- * 性质3: 对任何一棵二叉树, 若其有 n_0 个叶结点, n_2 个度为2的非叶结点, 则 $n_0 = n_2 + 1$

- * 性质4: 具有 n 个结点的完全二叉树的深度为 $\lceil \log_2(n+1) \rceil$ (即深度 $k: 2^{k-1} - 1 < n \leq 2^k - 1$)

(Tips: 理想平衡二叉树 (最下面一层结点不一定集中在最左边) 也适用性质4)

- * 性质5: 若将一棵有 n 个结点的完全二叉树从顶向下, 从左向右的结点依次编号 $1, 2, \dots, n$, 并称编号为 i 的结点为结点 i ($1 \leq i \leq n$), 则有以下位置关系:

- (1) 若 $i=1$, 结点 i 为根, 无父结点
- (2) 若 $i>1$, 则结点 i 的双亲为 $\lfloor i/2 \rfloor$
- (3) 若 $2i \leq n$, 则结点 i 的左子女为 $2i$; 同理其右子女为 $2i+1$
- (4) 结点 i 所在的层次为 $\lfloor \log_2 i \rfloor + 1$

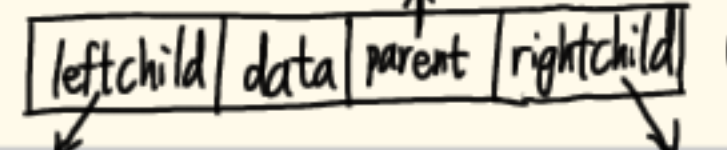
- 二叉树的存储

- 完全二叉树方便用数组表示, 自顶向下, 同一层自左向右编号 (一定连续) 即可存数组

一般二叉树也可以用数组表示, 即仿照完全二叉树, 有的结点存入对应编号的位置;

没有的 (虚结点) 位置空着 (可能造成大量的空间浪费)

- 二叉链表表示: 每个结点包含:  (两个指针指向子女)

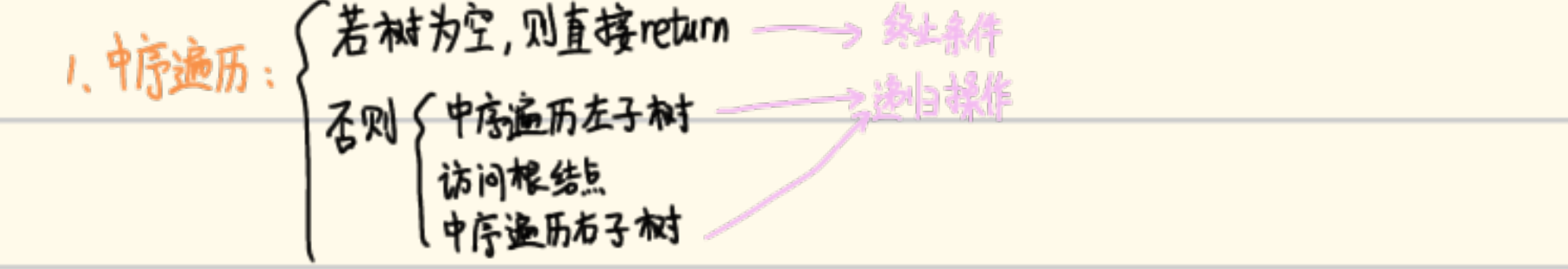
三叉链表表示: 每个结点包含:  (新增一个指针方便找父结点)

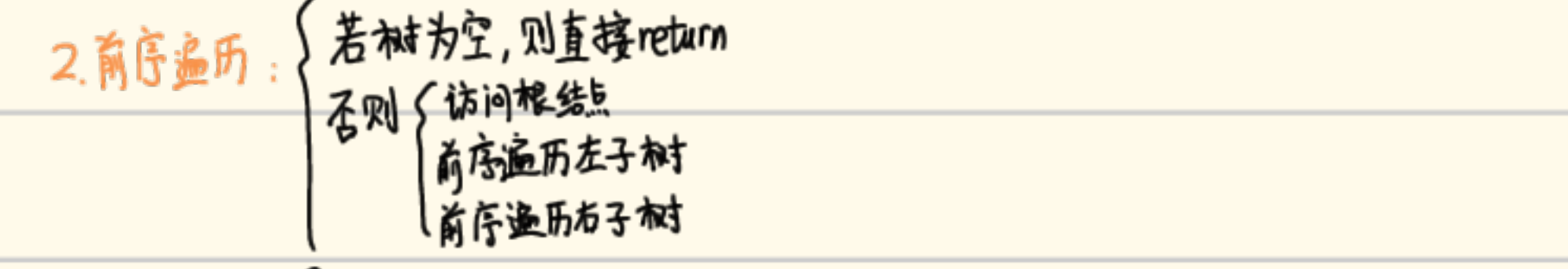


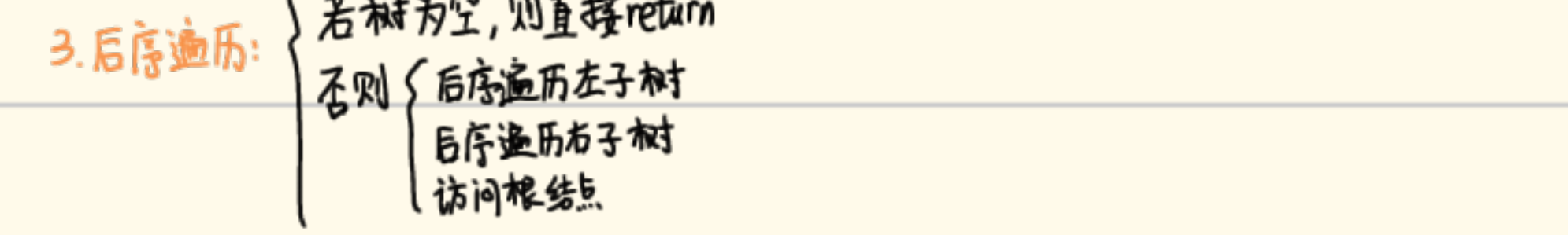
(此链表也可以用静态链表表示, 即用数组表示链表)

- 二叉树的遍历 (按某种次序访问二叉树中的所有结点, 不重且不漏)

(-) 深度优先 (前序 VLR, 中序 LVR, 后序 LRV, 其中 V 表示根)

1. 中序遍历: 

2. 前序遍历: 

3. 后序遍历: 

** 非递归方式前序遍历二叉树 (栈)

预先建立一个栈节点栈, 预存一个 NULL, 初始 P 指向根节点。

```
while (P != NULL) {  
    visit(P) // 访问P (前序先访根)  
    if (P->rightchild != NULL) S.push(P->rightchild); // 栈中预存右子节点 (若有), 留后手  
    if (P->leftchild != NULL) P = P->leftchild; // 若有左节点, 则深度优先调用  
    else S.pop(P); // 若无左节点, 说明当前前序访问对应右节点 (想高树), 故退栈 (已访问)
```

** 非递归方式中序遍历二叉树 (栈)

遍历指针 P 从根节点开始, 一路往左并将沿途节点进栈 (直到某一节点左孩子为空), 然后 (若栈不空) 退栈, 访问退出的节点, 并将遍历指针 P 置在其右节点。

* 需使用 do-while 语句, 较复杂:

```
do { while (P != NULL) { S.push(P); P = P->leftchild; }  
    if (!S.isEmpty()) { S.pop(P); visit(P); P = P->rightchild; }  
} while (P != NULL || !S.isEmpty())
```

** 非递归方式后序遍历二叉树 (栈)

构造一个结构体 (每个结点 + 标志位 L: 左子树返回了, 还有右子树 / R: 右子树返回了, 访问根节点)

* 逻辑: 每个节点进栈时均为 "TreeNode + L", 然后结点变到其左子树, 若无, 则栈顶的标志改为 R, 否则将其左子树根节点进栈 (循环操作)。栈顶为 "R" 时, 则将栈顶结点的右子树根入栈 (若有) (循环操作), 否则访问栈顶的节点并将其退栈。直到栈空。
// 同样需要 do-while 语句, 代码见教材。

(=) 广度优先 (层次序访问, 一层访问完后再看下一层)

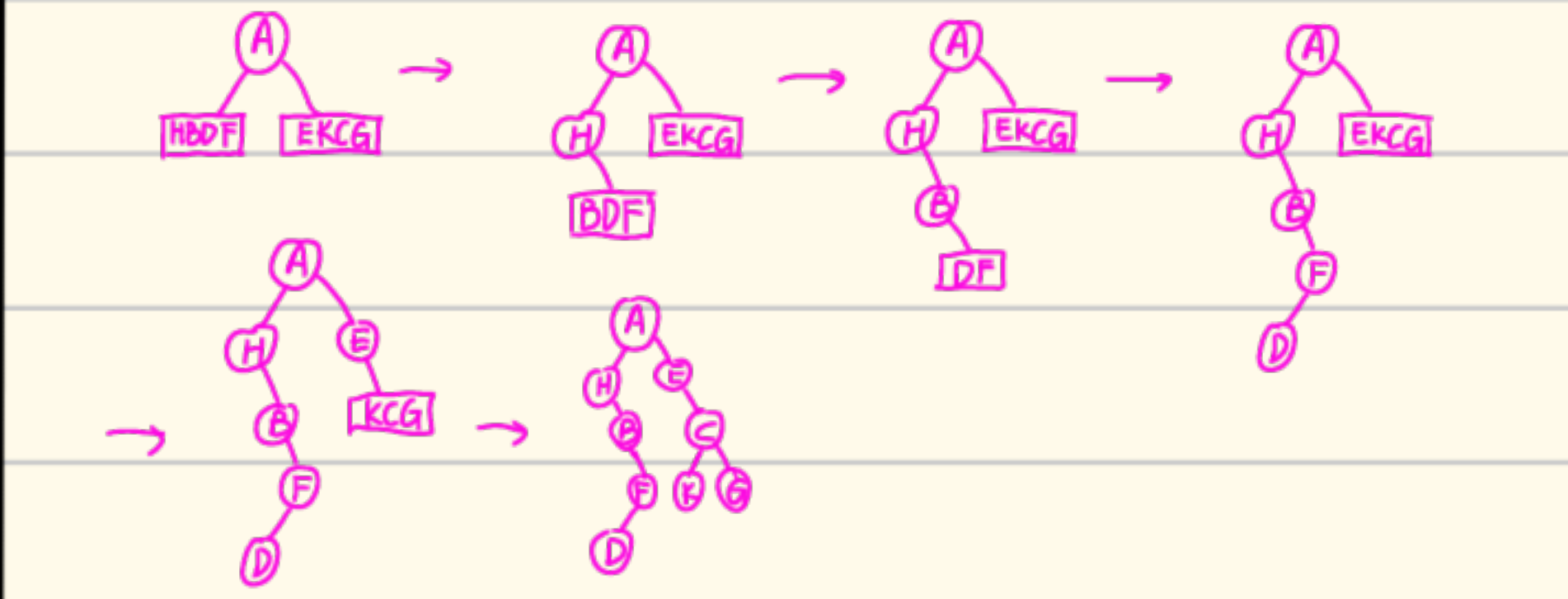
* 使用队列实现, 预建立队列并将树根进队

逻辑: 若队列不为空, 则访问队头并将其出队,

然后将其左、右节点 (若有) 依次进队 (类似: 杨辉三角的实现)

* 必考考点: 根据一棵二叉树的前序序列和中序序列, 唯一确定二叉树。

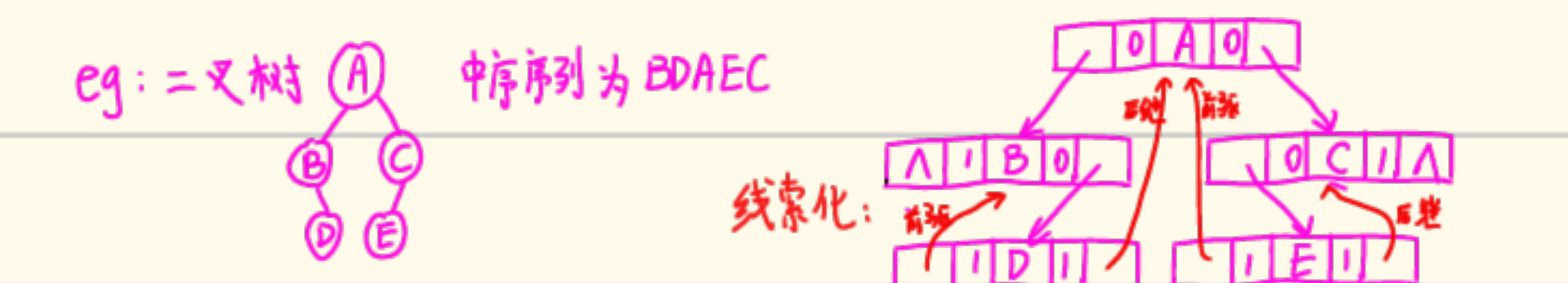
eg: 前序序列 AHBDFDECKG, 中序序列 HBDFAEKCG



• 线索二叉树

二叉树的前、中、后序遍历得到了关于结点的线性序列，故每个结点在对应的序列中都有前驱和后继（若无，则为序列头、尾结点），因此可以给每个结点加上指示前驱、后继信息的域，即为线索。（这样找前驱、后继都为 $O(1)$ 过程且不需栈辅助）

实现方法：充分利用树结点中原先存放空指针的地方（改造树结点）
在树结点中添加 ltag 和 rtag 标志位，指示 LeftChild 和 RightChild 域中存的是子还是前驱后继
* 规则：若 ltag=0，则 leftchild 域中存放的是表示左子女结点的指针；若 ltag=1，则 leftchild 域中存放的是表示该结点中序下前驱的线索。rtag 同理。



** 以下 (内) 代码实现皆以中序为例:

• 如何由某一节点 current 寻找后继结点

current → rtag	= 0	= 1
current → rightChild	不可能出现	无后继
= NULL	不可能出现	无后继
!= NULL	后继为 rightchild 为根的子树在中序下的第 1 个结点	后继即为 rightchild

• 如何由某一节点 current 寻找前驱结点

current → ltag	= 0	= 1
current → leftChild	不可能出现	无前驱
= NULL	不可能出现	无前驱
!= NULL	前驱为 leftchild 为根的子树在中序下的最后 1 个结点	前驱即为 leftchild

(代码以找后继为例)

```
辅助函数 Node *First(Node* current) { // 找子树在中序下的第 1 个结点
    Node* p = current; while (p->ltag == 0) p = p->leftChild;
    return p; }
```

```
主函数 Node *Next(Node* current) { // 找后继的函数
    Node* p = current->rightChild;
    if (p->rtag == 0) return First(p); else return p; }
```

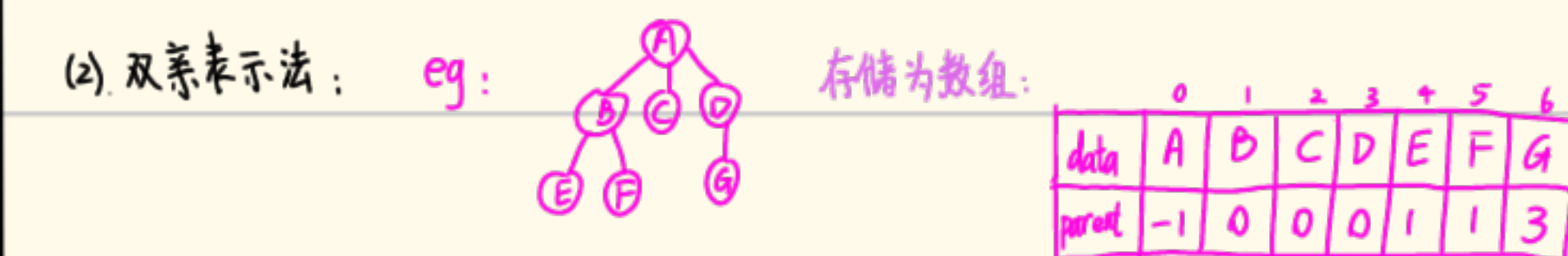
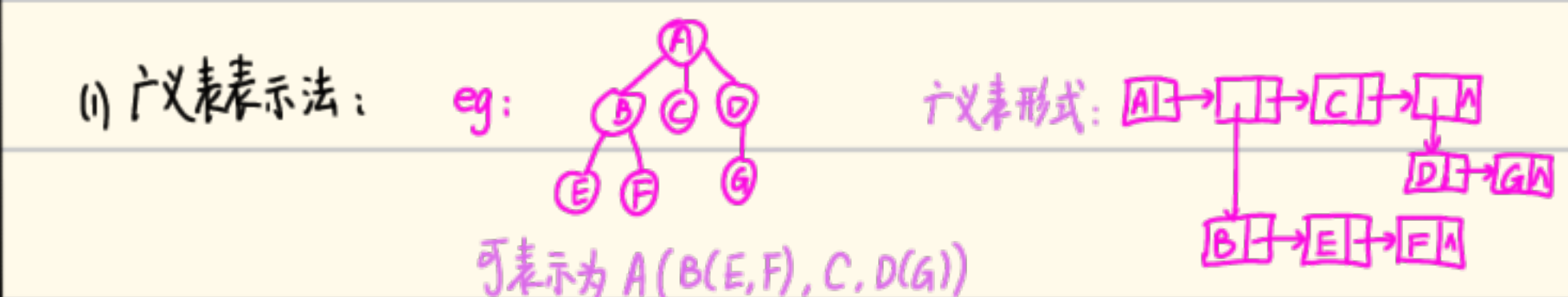
// 找前驱同理

** 前序/后序线索化二叉树: 将二叉树按前/后序线索化的规则与中序线索化同理

此时寻找当前结点的前驱/后继较复杂, 选择性掌握

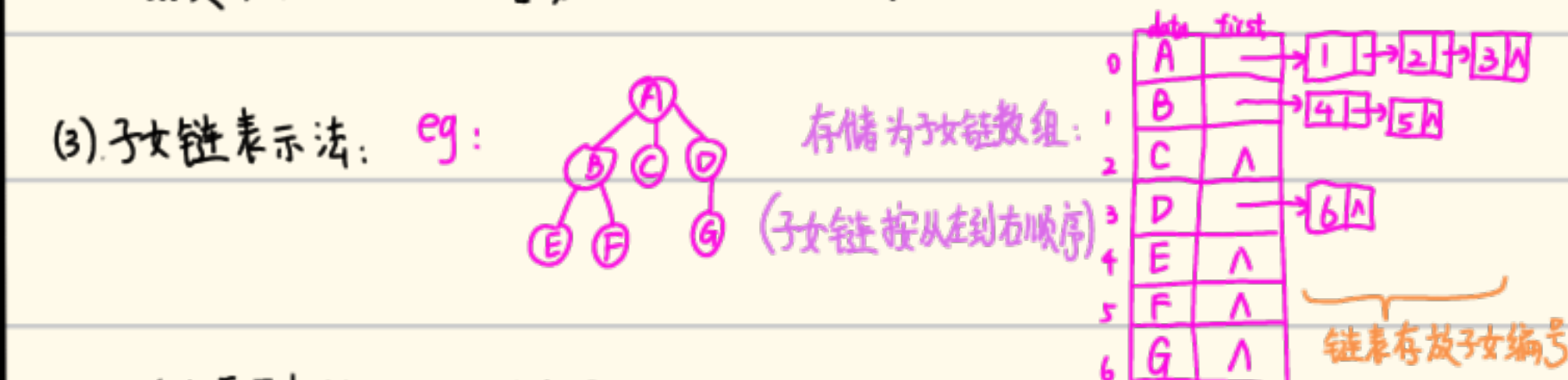
• 树与森林

• 一般的树的存储表示



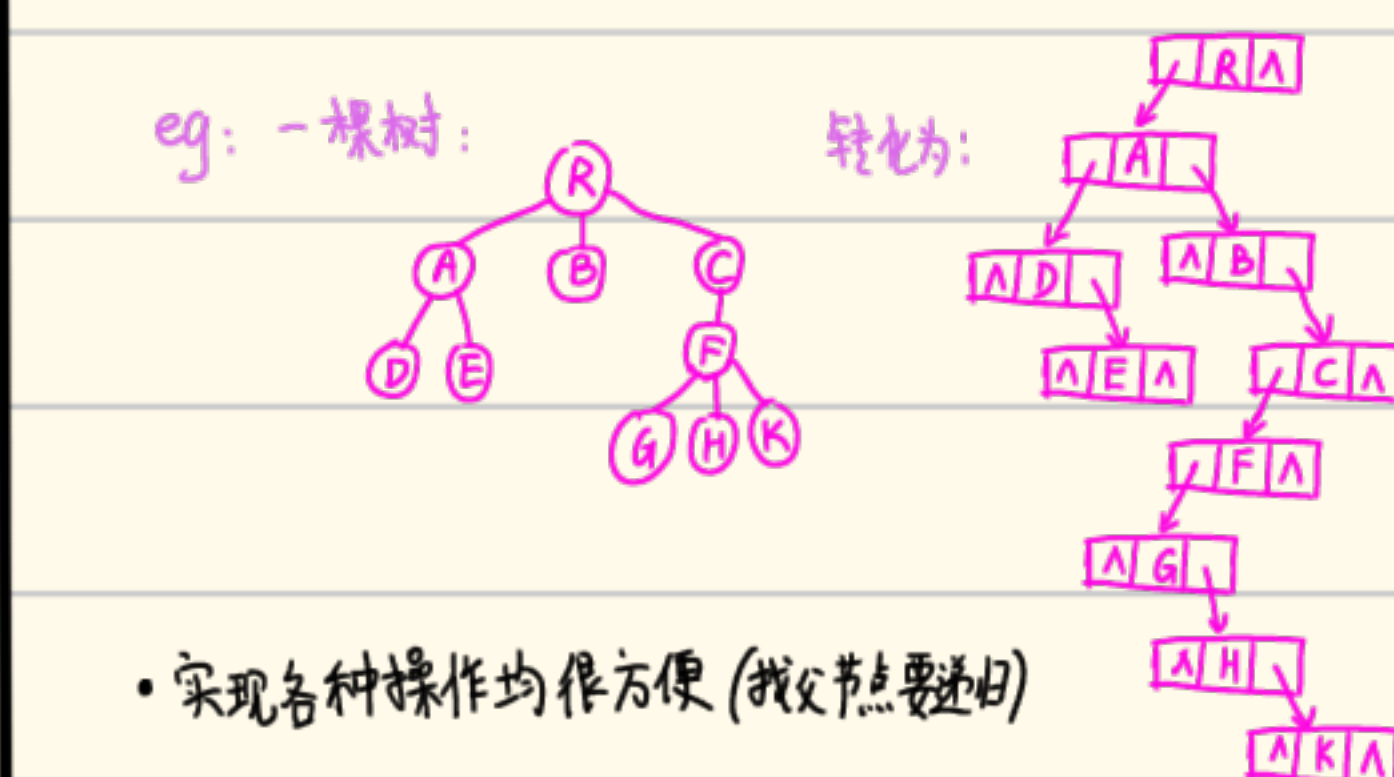
若不强调存储顺序, 则为方便可以按照层序 (广度优先搜索)

• 好处: 添加叶结点很容易 • 坏处: 删除非叶结点时, 会波及子女



• 适合需要频繁寻找子女的应用

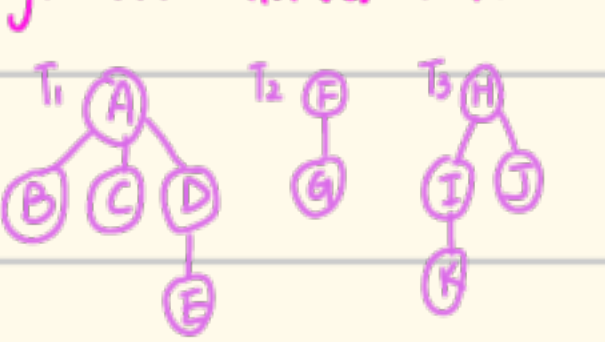
(4) 子女-兄弟链表表示: (最节省存储空间) 每个结点: firstChild/data/nextSibling



• 实现各种操作均很方便 (找父节点要递归)

* 此方法可将任意树转化为二叉树, 最常用

• 森林的表示 (用二叉树)

eg: 以下三棵树的森林:  ⇒ 转化为二叉树 (子女-兄弟链) ⇒ 根结点连起来即可



• 树的遍历

1. 树的深度优先遍历 (设树为 $T = \{r, T_1, T_2, \dots, T_n\}$)

先根遍历: 若 $T = \emptyset$ 返回, 否则访问 r , 然后依次访问 $T_1, T_2, \dots, T_n$

后根遍历: 若 $T = \emptyset$ 返回, 否则依次访问 $T_1, T_2, \dots, T_n$, 然后访问 r

* 不妨设 T 转化为子女-兄弟链 (即二叉树) 为 T' , T 的先根遍历和 T' 前序遍历结果一致

T 的后根遍历和 T' 中序遍历结果一致

2. 树的广度优先遍历 (设树为 $T = \{r, T_1, T_2, \dots, T_n\}$)

即使用队列, 层次化的输出 (根入队 → 队头出队访问 → 其所有子女依次入队)

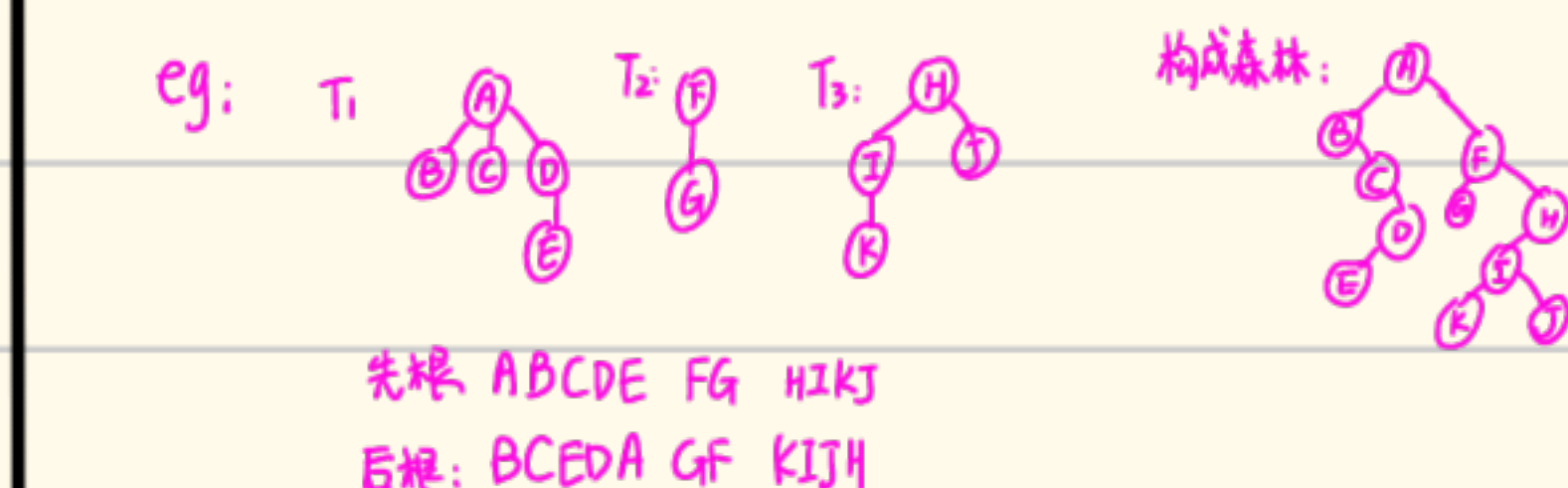
• 森林的遍历

1. 深度优先:

先根遍历: 若 $F = \emptyset$, 结束; 否则访问 T_1 的 root, 然后先根遍历 T_1 的所有子树

$T_{11}, T_{12}, \dots, T_{1k}$; 最后递归调用 $T_1 \rightarrow \text{rightchild}$ (即 $T_2, T_3, \dots, T_n$)

后根遍历: 同理, 把访问根节点的时机变一下即可



2. 广度优先:

依次访问各棵树的根结点, 然后依次遍历这些根结点的所有子女, ...

eg: 上述例子: AFH BCDGIJ EK

• 堆 (有顺序的完全二叉树)

最小堆: $K_i \leq K_{2i+1}$ & $K_i \leq K_{2i+2}$ (根一定不大于左右子女)

最大堆: $K_i \geq K_{2i+1}$ & $K_i \geq K_{2i+2}$ (根一定不小于左右子女)

(堆存储在下标从 0 开始的一维数组中, 下标可以用完全二叉树下标的性质来计算)

• 堆的操作: (以最小堆为例, 根 $\leq$ 左右子女)

• 下沉操作: 从结点 $\text{heap}[\text{start}]$ 开始, 一层层向下比较, 直至其比其左右子女中的较小者还小 / 其已无左右子女; 否则将较小者上移, 然后接着向下比。

• 上浮操作: 从结点 $\text{heap}[\text{start}]$ 开始, 一层层向上比较 (父结点 $i = (j-1)/2$); 直至父结点比其小 / 其下标已为 0; 否则将较大者下移, 然后接着向上比。

* 插入算法: 先把待插的结点放在 $\text{heap}[m]$ (即堆尾), 然后逐层上浮。

* 删除算法: 先将堆尾元素补到被删结点, 然后将此点结逐层下沉

(时间复杂度均为 $O(\log n)$)

• Huffman 树

• 路径长度: 从根节点到达树的每一个其他节点的路径长度即为该节点所在层次 $k-1$

显然, 总路径长度在节点数一致时, 树越平衡越小 (完全二叉树最优)

$$\text{即 } PL = \sum_{i=0}^{n-1} \lfloor \log_2(i+1) \rfloor$$

• 带权路径长度: 设 $\{w_1, w_2, \dots, w_n\}$, 其中 $w_i \geq 0$, 分别为二叉树 T 的 n 个叶节点的权

$$\text{则带权路径长度定义为 } WPL = \sum_{i=1}^n w_i \times l_i \quad (l_i \text{ 为对应叶节点层次 } k-1)$$

* WPL 最小的不一定是完全二叉树。

Huffman 树即为带权路径 WPL 达到最小的二叉树 (权值越大, 离根结点越近)

* Huffman 树的构造算法: (给定 n 个权值 $w_0, w_1, \dots, w_{n-1}$)

1. 先建立一个森林 $F = \{T_0, T_1, \dots, T_{n-1}\}$, 其中 T_i 为只带权值为 w_i 的根的二叉树 (无左右孩子)

2. 用最小堆存放这个森林, 然后重复执行步骤 3 直至堆中只存 1 棵扩充二叉树

3. 3.1 从堆中依次删除两次 $\text{heap}[0]$ (即: 权值最小的两棵扩充二叉树)

3.2 将这两棵的根节点作为 *parent (新根) 的 左、右孩子, ^(不要按顺序) parent 权值为两个根节点权值的和

3.3 将 *parent 插入最小堆

* Huffman 树的应用:

Huffman 编码 (最常用的最短, 且是前缀编码不会混淆)